

A Configurable System for Table Cell Extraction from Scanned Documents

Hamed Keldarreh

February 23, 2026

Abstract

Scanned documents containing tabular forms are common tasks in administrative and industrial workflows, yet manual transcription of table contents is costly and prone to error. This work presents a methodology for automatic extraction of individual table cells from document images. The system detects table grid lines using *OpenCV* morphological operations, derives cell bounding boxes from contours, and assigns consistent row and column indices via a reference-row alignment strategy that is robust to merged and missing cells. All behavior is controlled through an 18-parameter `TableConfig` dataclass, enabling adaptation to diverse table layouts and image qualities without the need to modify code. A companion validation tool computes per-table coverage, grid dimensions, and aggregate statistics, providing quantitative feedback for parameter tuning and quality assurance.

Contents

1	Introduction	3
2	Problem Formulation	3
2.1	Input and Output	3
2.2	Design Requirements	4
3	System Architecture	4
3.1	Core Components	4
3.1.1	BoxAnnotation	4
3.1.2	TableConfig	5
3.1.3	TableAnalysis	5
4	Methodology: 8-Step Extraction Algorithm	5
4.1	Step 1: Image Loading and Preprocessing	5
4.2	Step 2: Kernel Definition for Line Detection	6
4.3	Step 3: Vertical and Horizontal Line Detection	6
4.4	Step 4: Grid Formation by Line Combination	6
4.5	Step 5: Contour Detection and Cell Candidate Extraction	6
4.6	Step 6: Row Grouping via Vertical Proximity	7
4.7	Step 7: Column Assignment Using a Reference Row	7
4.8	Step 8: Annotation Creation and Output Generation	7
5	Implementation Details	8
5.1	Command-Line Interface and Batch Processing	8
5.2	Validation and Coverage Analysis	8

6	Interactive App	9
6.1	User Interface and Interaction	10
6.1.1	Configuration Panel	10
6.1.2	Single-Image Workflow	10
6.1.3	Preset Comparison	10
6.1.4	Batch Processing	11
7	Discussion and Future Work	11
7.1	Limitations	11
7.2	Potential Extensions	12

1 Introduction

Structured tables are prevalent in scanned forms such as medical questionnaires, insurance applications and administrative records, where each cell often corresponds to a specific semantic field. Manual transcription of these cells into downstream systems is laborious and susceptible to error. Thereby automated solutions that can reliably locate and index each cells are highly promising.

While a large line of research addresses table detection and recognition, many real-world deployments still rely on classical computer-vision techniques due to their transparency, low computational cost, and ease of deployment without training data. The analysis in this project represents such an approach where it implements an OpenCV-based pipeline for line-based tables.

The objectives of the system are:

1. to extract precise bounding boxes for all cells in a table image,
2. to assign each cell a unique row and column index, and
3. to output cell images as `cell-row-col.png` for subsequent (perhaps manual) process.

The implementation is based on the inscription and guiding principles provided by the tutorial notebook “Image-based Table Detection and Cell Recognition with OpenCV” which describes procedure for identifying table lines, detecting contours, and organizing cells into rows and columns using Python and OpenCV. The tutorial introduces the core ideas of binary thresholding, morphological line detection using structuring elements, contour extraction, and proximity-based grouping, which form the conceptual foundation of the implemented system.

Compared to the tutorial, the current project extends the pipeline into a general-purpose, configurable library with a separation between configuration (`TableConfig`), processing logic (`TableAnalysis`), and validation (`validate_extraction.py`). It also adds filtering logic, robust error handling, batch processing support, command-line interfaces, and quantitative coverage analysis.

2 Problem Formulation

2.1 Input and Output

Given an input image I containing a table with visible horizontal and vertical grid lines, the task is to:

- Compute a set of cell annotations

$$\mathcal{C} = \{(x_i, y_i, w_i, h_i, r_i, c_i)\}_{i=1}^N,$$

where (x_i, y_i) is the upper-left corner of cell i , w_i, h_i are width and height in pixels, and r_i, c_i are the row and column indices, respectively.

- A corresponding set of **cell image crops**, stored as PNG files at:

```
results/{document_name}/cell-{row}-{col}.png
```

where rows are indexed from top to bottom and columns from left to right, both starting at zero.

2.2 Design Requirements

The implementation targets the following requirements:

- **Accuracy:** High coverage of true cells with minimal false positives.
- **Robustness:** Tolerance to moderate noise, small scanning artifacts, and merged or missing cells.
- **Configurability:** At least 18 tunable parameters controlling preprocessing, line detection, filtering, and row/column assignment.
- **Scalability:** Ability to process thousands of documents in batch mode with progress tracking.

3 System Architecture

The system comprises several key components:

- `extract_cells.py`: core extraction engine implementing configuration, the 8-step algorithm, and batch processing.
- `TableConfig` (in `extract_cells.py`): Encapsulates configuration parameters controlling all stages of the pipeline, including image preprocessing, line detection, cell filtering, and row/column assignment. Type hints and default values make it suitable both for beginners and for automated tuning.
- `TableAnalysis` (in `extract_cells.py`): class that executes the table-extraction algorithm and provides helper methods for row grouping, column assignment, and result writing.
- `validate_extraction.py`: validation tool that analyzes output directories and reports per-table and aggregate metrics.

3.1 Core Components

The system is organized around three main Python components:

3.1.1 BoxAnnotation

`BoxAnnotation` is a simple dataclass specifying the geometry and identity of a single cell:

- `x, y`: absolute coordinates of the upper-left corner.
- `width, height`: absolute width and height in pixels.
- `classname`: string identifier in the format `cell-row-col`.

This standardized representation enables consistent downstream handling of cell locations and logical positions.

3.1.2 TableConfig

TableConfig centralizes all configurable aspects of the pipeline:

- **Image preprocessing:** `binary_threshold`.
- **Line detection:** `kernel_length_divisor`, `vertical_kernel_width`, `horizontal_kernel_height`, `erode_iterations`, `dilate_iterations`, `final_erode_iterations`.
- **Line combination weights:** `vertical_line_weight`, `horizontal_line_weight`.
- **Cell filtering:** `min_cell_width`, `max_cell_width`, `min_cell_height`, `max_cell_height`, `filter_by_area_ratio`, `min_area_ratio`, `max_area_ratio`.
- **Row/column assignment:** `row_grouping_tolerance`, `column_center_tolerance`.

3.1.3 TableAnalysis

TableAnalysis class Implements the full extraction pipeline. Its key methods are:

- `process(filepath) -> List[BoxAnnotation]`: runs the algorithm on a single image.
- `group_into_rows(boxes)`: groups bounding boxes into rows.
- `assign_columns(rows)`: assigns column indices using a reference-row strategy.
- `write_results(boxes, filepath, outputdir)`: crops and saves cell images.

A command-line interface wraps this functionality to process entire directories of images and store results in a structured output tree.

4 Methodology: 8-Step Extraction Algorithm

The core algorithm in `TableAnalysis.process` follows an 8-step pipeline derived from the tutorial and extended for robustness.

4.1 Step 1: Image Loading and Preprocessing

The input image is loaded in grayscale using OpenCV; failure to load triggers a warning and early return. Binary thresholding is applied using Otsu's method with a configurable initial threshold, followed by inversion so that table lines become white on a black background:

- Load grayscale: `cv2.imread(..., cv2.IMREAD_GRAYSCALE)`.
- Threshold: `cv2.threshold(img, binary_threshold, 255, THRESH_BINARY | THRESH_OTSU)`.
- Invert: `img_bin = 255 - img_bin`.

This representation simplifies subsequent line detection operations.

The `process` method reads the input image in grayscale using OpenCV:

```
img = cv2.imread(str(filepath), cv2.IMREAD_GRAYSCALE)
```

If loading fails, the method prints a warning and returns an empty list of annotations, ensuring robustness against missing or corrupted files.

Binary thresholding uses a configurable threshold combined with Otsu's method:

```
_, img_bin = cv2.threshold(
    img,
    config.binary_threshold,
    255,
    cv2.THRESH_BINARY | cv2.THRESH_OTSU
)
img_bin = 255 - img_bin # invert: white lines on black
                        background
```

The inversion step simplifies subsequent morphological operations by treating lines as foreground.

4.2 Step 2: Kernel Definition for Line Detection

The algorithm constructs three structuring elements (kernels):

- A **vertical kernel** of shape (vertical_kernel_width, kernel_length) for detecting vertical lines.
- A **horizontal kernel** of shape (kernel_length, horizontal_kernel_height) for detecting horizontal lines.
- A **cleanup kernel** of size 2×2 for later morphological refinement.

`kernel_length` is derived as `image_width // kernel_length_divisor`, linking kernel size to image resolution. Smaller divisors yield longer kernels and are suitable for detecting long, thin lines in dense tables.

4.3 Step 3: Vertical and Horizontal Line Detection

Vertical and horizontal lines are detected separately via erosion followed by dilation:

- Vertical lines: erode and dilate with the vertical kernel.
- Horizontal lines: erode and dilate with the horizontal kernel.

Configurable iteration counts (`erode_iterations`, `dilate_iterations`) control the aggressiveness of the operations, enabling adaptation to varying line thickness and noise levels.

4.4 Step 4: Grid Formation by Line Combination

The vertical and horizontal line images are combined using a weighted sum:

$$\text{grid} = w_v \cdot V + w_h \cdot H$$

implemented via `cv2.addWeighted`, where w_v and w_h are *vertical_line_weight* and *horizontal_line_weight* respectively.

4.5 Step 5: Contour Detection and Cell Candidate Extraction

Contours are extracted from the grid image using `cv2.findContours`. Each contour is converted into a bounding rectangle (x, y, w, h) using `cv2.boundingRect`.

Candidate boxes are then filtered according to the size and area constraints defined in `TableConfig`:

- Width and height must lie within `[min_cell_width, max_cell_width]` and `[min_cell_height, max_cell_height]`.

- If `filter_by_area_ratio` is enabled, the area ratio $(wh)/\text{image_area}$ must fall within `[min_area_ratio, max_area_ratio]`.

Boxes passing all filters constitute the set of potential cells. If no valid boxes remain, a warning is logged and processing stops for that image.

4.6 Step 6: Row Grouping via Vertical Proximity

Row grouping is handled by the `group_into_rows` method. Boxes are first sorted by their vertical coordinate y , and the mean height of all boxes is computed. A tolerance value is calculated as:

$$\text{tolerance} = \text{mean_height} \times \text{row_grouping_tolerance}$$

Two boxes are placed in the same row if the absolute difference between their y positions is less than or equal to this tolerance. Each row is then sorted left-to-right by x , producing a list of ordered rows.

4.7 Step 7: Column Assignment Using a Reference Row

To accommodate merged cells and incomplete rows, columns are assigned using a reference-row approach implemented in `assign_columns`.

1. The row with the maximum number of boxes is selected as the reference row.
2. For each box in the reference row, a horizontal “center” value is computed and collected into a sorted list of column centers.
3. For every row, each box’s center is compared with these column centers, and the box is assigned to the nearest center, producing a mapping from column index to list of boxes.

The hyper-parameter `column_center_tolerance` influences the effective spacing of centers and thereby the sensitivity of column alignment. This strategy preserves column consistency even when some rows lack cells in particular columns or contain horizontally merged cells.

4.8 Step 8: Annotation Creation and Output Generation

Finally, for each row and column index, one or more boxes are wrapped in `BoxAnnotation` instances with class names of the form `cell-row-col`. The `write_results` method then:

1. Loads the original color image.
2. Clamps each box’s coordinates to the image boundaries to avoid indexing errors.
3. Crops the cell region and writes it to disk under the appropriate `results/{document}/` subdirectory using `cv2.imwrite`.

Exceptions encountered during cropping or writing are caught and reported as warnings, allowing processing of other cells and documents to continue uninterrupted.

5 Implementation Details

5.1 Command-Line Interface and Batch Processing

The `main` function in `extract_cells.py` accepts an optional input directory and output directory, defaulting to `.tables` and `.results` respectively. It enumerates all `.png` and `.jpg` files, sorts them for deterministic processing, and iterates over them with a `tqdm` progress bar. For each image, it instantiates `TableAnalysis` with the desired configuration, calls `process`, and passes the resulting annotations to `write_results`.

Example usage includes both default and custom configurations:

```
# Default configuration
python extract_cells.py tables results

# Programmatic configuration for dense tables
from extractcells import TableConfig, main
config = TableConfig(
    row_grouping_tolerance=0.3,
    column_center_tolerance=3,
    kernel_length_divisor=80,
    min_cell_width=15,
    min_cell_height=15,
)
main("tables", "results", config)
```

5.2 Validation and Coverage Analysis

`validate_extraction.py` provides automated evaluation of extraction quality. The `analyze_results` function:

1. Scans each table subdirectory under a given results directory.
2. Uses a regular expression to parse row and column indices from filenames matching `cell-<row>-<col>.png`.
3. Computes:
 - `total_cells`,
 - maximum observed row and column,
 - expected cells assuming a complete grid,
 - coverage percentage defined as `total_cells / expected_cells * 100` (if expected cells > 0).

Tables are categorized into quality statuses according to coverage thresholds:

EXCELLENT $\geq 95\%$, GOOD $\geq 80\%$, FAIR $\geq 60\%$, POOR otherwise, EMPTY for zero cells.

The `print_analysis` function summarizes per-table metrics and aggregates total cells, average coverage, and coverage range.

The `identify_issues` function returns a list of tables whose coverage falls below a configurable threshold (default 80%), guiding users toward parameter tuning and inspection of problematic documents.

6 Interactive App

In this section we describe a Streamlit-based web application that demonstrate the modules and functionalities for image-based table cell extraction. The app provides interactive parameter control, visual feedback, and batch processing, making it suitable both as a research prototype and as a portfolio artifact for document understanding systems.

As outlined in the preceding sections, the underlying extraction engine in `extract_cells.py` implements a classical OpenCV pipeline to detect table grid lines, segment individual cells, and assign row/column indices. The Streamlit application (`app.py`) wraps this engine and the validation logic from `validate_extraction.py` into an interactive interface for non-programmers.

Figure 1 illustrates the main user interface with a configuration sidebar and two primary workflows: single-image analysis and batch processing.

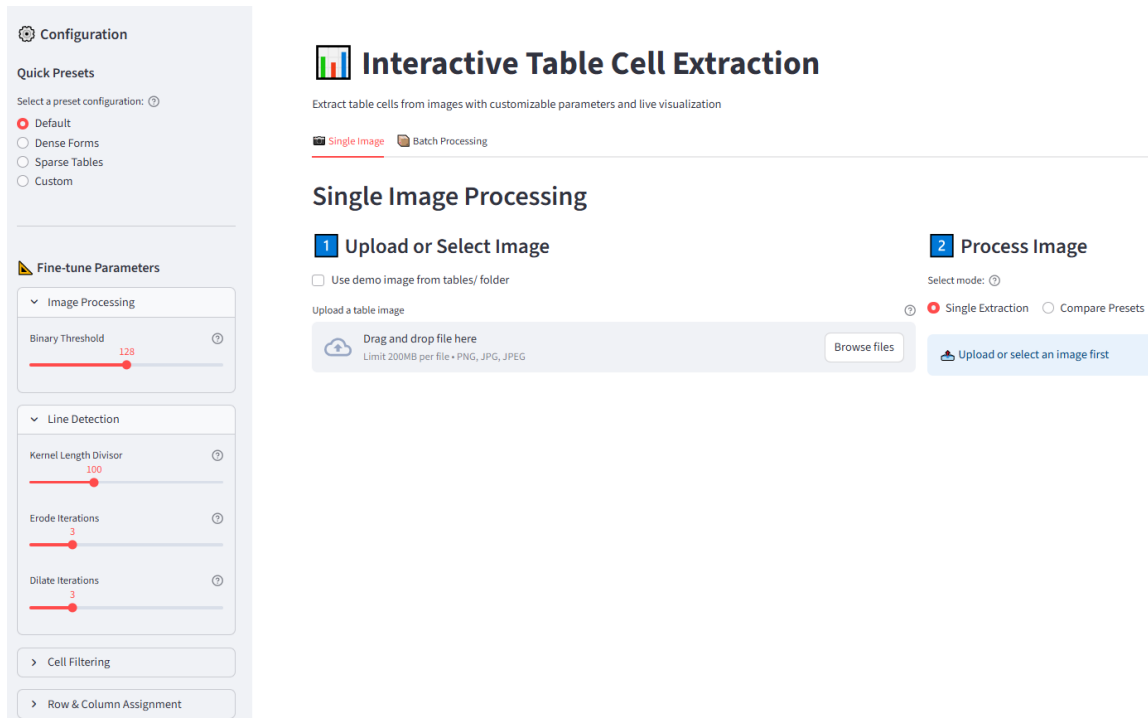


Figure 1: Main User Interface: sidebar with configuration controls (left) and tabs for Single Image and Batch Processing (right).

Figure 2 summarises the data flow: browser requests are handled by Streamlit, which in turn calls the extraction and validation modules and returns annotated images and metrics.

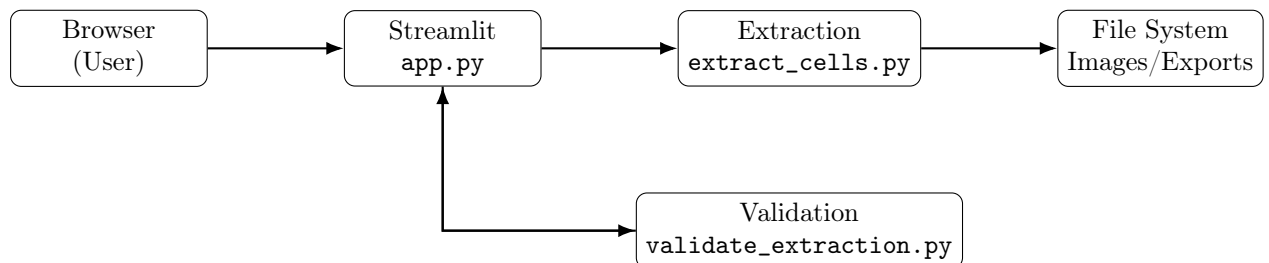


Figure 2: System architecture: the browser interacts with the Streamlit app, which calls the extraction and validation modules and reads/writes files.

6.1 User Interface and Interaction

6.1.1 Configuration Panel

The left sidebar comprises:

- **Quick Presets:** *Default*, *Dense Forms*, *Sparse Tables*, and *Custom*, each corresponding to predefined `TableConfig` settings.
- **Sliders:** For key parameters such as binary threshold, kernel length divisor, erosion/dilation iterations, minimum and maximum cell size, and row/column tolerances.

We have summarized the default preset values for three modes in table 1 .

Parameter	Default	Dense Forms	Sparse Tables
kernel_length_divisor	100	80	120
row_grouping_tolerance	0.5	0.3	0.7
column_center_tolerance	4	3	6
min_cell_width	10	15	20
max_cell_width	1000	800	1500

Table 1: Preset configurations exposed in the sidebar.

6.1.2 Single-Image Workflow

The “Single Image” tab supports:

- Selecting a demo image from a `tables/` directory or uploading a Png/Jpg.
- Choosing either “Single Extraction” or “Compare Presets” mode.
- Triggering extraction, which yields:
 - An annotated image with green rectangles around detected cells.
 - Numerical metrics: total cells, grid size, coverage, and status.
 - A gallery of cropped cells, sortable by row/column or size.

Figure 3 shows a typical annotated image, and Figure 4 illustrates the cell gallery.

The app also exposes export options for the current image: a ZIP containing all cells, a CSV of coordinates, a JSON summary of metrics and parameters, and a PNG of the annotated image.

6.1.3 Preset Comparison

In “Compare Presets” mode, the same image is processed with three presets `Default`, `Dense` and `Sparse` layout. The app presents, for each preset:

- An annotated image.
- Metrics: cell count, grid dimensions, coverage, and status.

A comparison table is generated, and the preset with highest coverage is recommended. Figure 5 shows a schematic side-by-side comparison.

	Mitglied	Ehegatte/Lebenspartner	Kind	Kind
Name				
Vorname				
Geb. - Datum				
KVNR				
Verwandtschaftsverhältnis		Ehegatte	Leibliches Kind	
Krankenkasse	NAME KRANKENKASSE	☐ BKK ☐ Fremdkasse Name:	☐ BKK ☐ Fremdkasse Name:	☐ BKK ☐ Fremdkasse Name:
Lebt mit dem Mitglied im gemeinsamen Haushalt?		☐ Ja	☐ Ja	☐ Ja
Sind Einnahmen vorhanden?	☒ Ja	☐ Ja	☐ Ja	☐ Ja
Welche Einnahmearten erhalten Sie?	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug ☐ Keine Einnahmen	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug ☐ Keine Einnahmen	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug ☐ Keine Einnahmen
Haben Sie neben der Rente / neben den Leistungen zur Sicherung des Lebensunterhalts noch weitere Einnahmen?	☐ Ja	☐ Ja	☐ Ja	☐ Ja
Liegt eine chronische Erkrankung vor? Wenn ja, bitte Bescheinigung einreichen.	☒ Ja	☐ Ja	☐ Ja	☐ Ja
Sind Sie Teilnehmer am Behandlungsprogramm DMP?	☐ Ja	☐ Ja	☐ Ja	☐ Ja
Haben Sie einen Schwerbehindertenausweis mit einem Grad der Behinderung von mindestens 60?	☐ Ja	☐ Ja	☐ Ja	☐ Ja
Liegt bei Ihnen ein Pflegegrad von mindestens 3 vor?	☐ Ja	☐ Ja	☐ Ja	☐ Ja

Figure 3: Annotated table image with detected cell bounding boxes.

6.1.4 Batch Processing

The “Batch Processing” tab extends the app pipeline to multiple images. Users may:

- Process all images from the `tables/` folder or upload a ZIP archive.
- Monitor progress via a progress bar.
- Inspect a summary table listing, per image: cell count, grid size, coverage, status, and any error messages.
- Export all cells as a single ZIP and the summary as a CSV file.

Figure 6 shows an example batch summary view.

7 Discussion and Future Work

In a nutshell, we have reviewed a Python implementation for automatic table-cell extraction from scanned document images. The system leverages classical OpenCV morphological operations to detect table grids, contour analysis for cell candidate selection, and a proximity-based method to assign row and column indices. A validation framework provides quantitative metrics that guide parameter tuning and ensure configurability and robustness.

7.1 Limitations

Despite its robustness, the approach has inherent limitations:

- It assumes **line-based tables** with reasonably visible grid lines; heavily stylized or borderless tables may not be handled correctly.

	Mitglied	Ehegatte/Lebenspartner	Kind	Kind	Name				
cell-0-0 (412×116px)	cell-0-1 (407×116px)	cell-0-2 (424×117px)	cell-0-3 (407×117px)	cell-0-4 (409×116px)	cell-1-0 (412×116px)	cell-1-1 (408×117px)	cell-1-2 (423×116px)	cell-1-3 (408×116px)	cell-1-4 (408×117px)
Vorname					Geb. - Datum				
cell-2-0 (412×116px)	cell-2-1 (407×115px)	cell-2-2 (424×116px)	cell-2-3 (408×115px)	cell-2-4 (409×116px)	cell-3-0 (413×115px)	cell-3-1 (407×115px)	cell-3-2 (424×115px)	cell-3-3 (407×114px)	cell-3-4 (409×114px)
KvNR					Versandtschaftsverhältnis		Ehegatte	Leibliches Kind	
cell-4-0 (411×116px)	cell-4-1 (408×116px)	cell-4-2 (424×117px)	cell-4-3 (407×117px)	cell-4-4 (409×116px)	cell-5-0 (412×117px)	cell-5-1 (408×116px)	cell-5-2 (423×117px)	cell-5-3 (408×117px)	cell-5-4 (408×117px)
Krankenkasse	NAME KRANKENKASSE	☐ BKK ☐ Fremdkasse Name:	☐ BKK ☐ Fremdkasse Name:	☐ BKK ☐ Fremdkasse Name:	Lebt mit dem Mitglied im gemeinsamen Haushalt?		☐ Ja	☐ Ja	☐ Ja
cell-6-0 (412×116px)	cell-6-1 (407×116px)	cell-6-2 (424×116px)	cell-6-3 (408×117px)	cell-6-4 (409×117px)	cell-7-0 (412×116px)	cell-7-1 (407×116px)	cell-7-2 (424×117px)	cell-7-3 (407×117px)	cell-7-4 (409×116px)
Sind Einnahmen vorhanden?	☒ Ja	☐ Ja	☐ Ja	☐ Ja	Welche Einnahmenarten erhalten Sie?	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug	☐ Arbeitslosengeld II ☐ Rente ☐ Versorgungsbezug
cell-8-0 (412×116px)	cell-8-1 (408×117px)	cell-8-2 (424×117px)	cell-8-3 (407×117px)	cell-8-4 (409×117px)	cell-9-0 (413×262px)	cell-9-1 (409×263px)	cell-9-2 (425×262px)	cell-9-3 (409×264px)	cell-9-4 (409×263px)
Haben Sie neben der Rente / Rente die Leistungen zur Sicherung des Lebensunterhalts noch weiteren Einnahmen?	☐ Ja	☐ Ja	☐ Ja	☐ Ja	Liegt eine chronische Erkrankung vor? Wenn ja, bitte Beschreibung einreichen.	☒ Ja	☐ Ja	☐ Ja	☐ Ja
cell-10-0 (413×143px)	cell-10-1 (408×143px)	cell-10-2 (425×143px)	cell-10-3 (407×143px)	cell-10-4 (409×143px)	cell-11-0 (414×116px)	cell-11-1 (409×116px)	cell-11-2 (424×117px)	cell-11-3 (407×116px)	cell-11-4 (408×117px)
Sind Sie Teilnehmer an Behandlungsprogrammen (DMP)?	☐ Ja	☐ Ja	☐ Ja	☐ Ja	Haben Sie einen Schwerbehindertenausweis mit einem Grad der Behinderung von mindestens 50?	☐ Ja	☐ Ja	☐ Ja	☐ Ja
cell-12-0 (412×116px)	cell-12-1 (408×118px)	cell-12-2 (424×117px)	cell-12-3 (407×118px)	cell-12-4 (409×117px)	cell-13-0 (413×142px)	cell-13-1 (408×143px)	cell-13-2 (425×143px)	cell-13-3 (407×144px)	cell-13-4 (409×143px)
Liegt bei Ihnen ein Pflege- grad von mindestens 3 vor?	☐ Ja	☐ Ja	☐ Ja	☐ Ja					
cell-14-0 (413×118px)	cell-14-1 (408×118px)	cell-14-2 (424×118px)	cell-14-3 (407×118px)	cell-14-4 (409×117px)					

Figure 4: A screenshot of the extracted cells gallery. Sorting can also be tuned.

- It presumes that tables are roughly **upright**; significant rotation or skew would require prior deskewing.
- Complex patterns of merged cells, especially when cells span multiple rows or columns irregularly, may lead to imperfect column alignment or gaps in the logical grid.
- The method does not include OCR; extracting text from cells requires integration with additional tools such as Tesseract or other OCR engines.

7.2 Potential Extensions

Future work could extend the system with rotation detection and correction, improved handling of complex merged cells, integration of OCR for end-to-end table digitization, and potential deep-learning components for borderless or highly irregular tables. Nonetheless, the current implementation already offers a strong, interpretable baseline for line-based table extraction and a valuable educational resource for learners exploring computer-vision techniques for document analysis.

Another solution that could be pursued is based on deep learning neural network: one can replace the OpenCV grid detection with a learned model while reusing your existing post-processing. To be more specific, first we build a labeled dataset of table images, using the current OpenCV pipeline plus some manual correction to get cell boxes or line masks. Then we train a segmentation model (e.g., U-Net) to predict table lines or cell regions. The full incorporation of this architecture necessitates deeper evaluation and additional analysis.

